

دليل أوامر Git & GitHub — العمل الجماعي على مشروع واحد

(1) الإعداد الأولي (مرة وحدة بس)

```
bash
git config --global user.name "اسمك"
git config --global user.email "بريدك@example.com"
```

بتحدد مين صاحب كل commit.

```
bash
git init
```

بتحول أي مجلد لمستودع (repository) git — مستخدمة إذا عم تبلش مشروع جديد من الصفر.

```
bash
git clone <رابط المستودع>
```

بتنزل نسخة كاملة من مشروع موجود على GitHub لجهازك (مع كل التاريخ والفروع).

(2) الأوامر الأساسية اليومية

```
bash
git status
```

بتوريك شو تغيّر بالمشروع (ملفات جديدة، معدّلة، متعارضة...).

```
bash
git add <اسم الملف>
git add . # تضيف كل الملفات المتغيرة دفعة وحدة
```

بتحضّر (stage) التعديلات قبل ما تعملها commit.

```
bash
git commit -m "رسالة توضح شو عملت"
```

بتسجل التعديلات يلي حصّرتها (staged) بنقطة ثابتة بتاريخ المشروع.

```
bash
git log
git log --oneline      # نسخة مختصرة، سطر لكل commit
git log --oneline -5   # بس 5 commits آخر
git log --oneline --graph # مع رسم بياني يوضح الفروع
```

بتوريك تاريخ ال commits.

```
bash
git diff
```

بتوريك بالضبط شو تغيّر بالملفات (سطر سطر) قبل ما تعمل add.

```
bash
git diff --staged
```

بتوريك شو محصّر (staged) وجاهز لل commit.

3) العمل مع GitHub (المستودع البعيد / remote)

```
bash
git remote -v
```

بتوريك روابط المستودع البعيد (origin) المرتبط بمشروعك.

```
bash
git remote add origin <رابط المستودع>
```

بتربط مجلدك المحلي بمستودع على GitHub (لما تبش مشروع جديد محلياً).

```
bash
git push origin <اسم الفرع>
git push upstream إذا الفرع الحالي مرتبط أصلاً بـ #
```

بترفع ال commits المحلية لـ GitHub.

E056 bash git pull origin <اسم الفرع> git pull	بتجيب آخر تعديلات من GitHub وتدمجها مباشرة بفرعك الحالي (= merge + fetch بأمر واحد).
E056 bash git fetch origin	بتجيب آخر تعديلات من GitHub بدون ما تدمجها بفرعك — بس تحدّث معلوماتك عن الفروع البعيدة، وانت تقرر شو بدك تعمل فيها بعدين.
4) العمل بالفروع (Branches)	
E056 bash git branch	بتوريك كل الفروع المحلية، والفرع الحالي معلّم بـ *
E056 bash git branch -a	بتوريك كل الفروع، محلية وبعيدة.
E056 bash git branch <اسم الفرع الجديد>	بتنشئ فرع جديد (بدون ما تنقل لك عليه).
E056 bash git checkout <اسم الفرع>	بتنقلك لفرع موجود.
E056 bash git checkout -b <اسم الفرع الجديد>	

بتنشئ فرع جديد **وتنقلك عليه** بأمر واحد — أكثر أمر مستخدم لما تبlish شغلة جديدة (feature).

```
bash
git switch <اسم الفرع>
git switch -c <اسم الفرع الجديد>
```

أوامر أحدث وأوضح بديلة عن `checkout` (نفس الوظيفة تقريباً).

```
bash
git branch -d <اسم الفرع>
```

بتحذف فرع محلي (بس إذا كان مدموج already، وإلا بترفض).

```
bash
git branch -D <اسم الفرع>
```

بتحذف فرع محلي بالقوة، حتى لو مش مدموج.

```
bash
git push origin --delete <اسم الفرع>
```

بتحذف فرع من GitHub.

```
bash
git push -u origin <اسم الفرع>
```

بترفع فرع جديد لأول مرة على GitHub وتربطه (upstream) بفرعك المحلي، عشان بعد هيك يكفي تكتب `git push` بس.

5) دمج الفروع (Merge)

```
bash
git checkout main
git merge <اسم الفرع>
```

بتمدج فرع ثاني جوا الفرع الحالي (هون مثلاً عم تدمج فرع جوا `main`).

لو صار تعارض (**conflict**):

بيحث علامات بالملف Git:

```
E056
<<<<<< HEAD
الكود متاع الفرع الحالي
=====
الكود متاع الفرع الثاني
>>>>>> اسم الفرع الثاني
```

تفتح الملف يدوياً، تقرر شو تخلي (تدمج الاثنين، أو تختار وحدة)، وتمسح الثلاثة علامات بالكامل.
تأكد ما ضلت علامات:

```
E056
bash
grep -n "<<<<<<||=====||>>>>>>" <اسم الملف>
```

تعلم جيت إنك حليت التعارض:

```
E056
bash
git add <اسم الملف>
```

تكمل الدمج:

```
E056
bash
git commit
```

(اختياري) تدفع التعديل:

```
E056
bash
git push
```

```
E056
bash
git merge --abort
```

بتلغي عملية الدمج بالكامل وترجع الملفات لحالتها قبل ما تبlish merge (مفيدة إذا التعارض كبير وبدك تبlish من جديد).

6) تحديث فرعك بآخر شغل من main (بدون ما تخسر شغلك)

```
bash
git checkout <فرعك>
git fetch origin
git merge origin/main
```

بتجيب آخر شغل من `main` وتدمجه جوا فرعك — أفضل شي تعمله بشكل دوري وانت شغال على فرع طويل، عشان تتجنب تعارضات كبيرة بالآخر.

```
bash
git rebase main
```

بديل عن `merge` — بياخد ال commits متاعك ويعيد "تشغيلها" فوق آخر نسخة من `main`، بدل ما يعمل commit دمج إضافي. بينتج تاريخ أنظف، بس أصعب شوي بحل التعارضات (لو صارت). تجنبها إذا الفرع مشترك مع ناس تانيين.

7) Pull Requests (على موقع GitHub)

الخطوات:

ادفع فرعك: `git push -u origin <اسم الفرع>`
روح لصفحة المستودع على GitHub → رح يظهرلك زر **"Compare & pull request"** تلقائياً.
اختار: `compare` ← `base: main`: فرعك
اكتب عنوان ووصف واضح لشو عملت.
دوس **"Create pull request"**.
لو في تعارض، GitHub رح يوريك زر **"Resolve conflicts"** تحل فيه من المتصفح مباشرة (نفس مبدأ الحل اليدوي).
بعد ما زملاءك يراجعو (review) ويوافقو، دوس **"Merge pull request"**.

8) أوامر مفيدة للطوارئ (تراجع عن أخطاء)

```
bash
git checkout -- <اسم الملف>
```

بتلغي كل التعديلات على ملف معين وترجعه لآخر commit (قبل `git add`).

```
bash
git reset <اسم الملف>
```

بتشيل ملف من ال staging area (بعد `git add`) بدون ما تلغي تعديلاتك.

<pre>bash git reset --soft HEAD~1</pre>													
بتلغي آخر commit، بس تخلي التعديلات محضّرة (staged) — مفيدة لو غلطت برسالة ال commit أو بدك تعدل عليه.													
<pre>bash git reset --hard HEAD~1</pre>	⚠ بتلغي آخر commit وكل التعديلات فيه نهائياً — استخدمها بحذر شديد.												
<pre>bash git revert <hash الكوميٲ></pre>	بتنشئ commit جديد يلغي تأثير commit سابق، بدون ما تحذف تاريخ المشروع — أأمن من <code>reset</code> لو الكوميٲ already مدفوع (pushed) ومشارك مع فريق.												
<pre>bash git stash</pre>	بتخبي التعديلات الحالية يلي ما لسا عملتلها commit (مؤقتاً)، عشان تقدر تبدل فرع بسرعة بدون ما تخسرها.												
<pre>bash git stash pop</pre>	بترجع آخر تعديلات خبيتها بـ <code>stash</code>.												
٩) أفضل ممارسات للعمل الجماعي													
<table> <tr> <th data-bbox="535 1606 673 1669">الممارسة</th><th data-bbox="535 1606 673 1669">ليش مهمة</th></tr> <tr> <td data-bbox="1161 1774 1624 1837">كل واحد يشتغل على فرع خاص (<code>feature/اسمك-المهمة</code>) يمنع تعارض شغل الكل مع بعض بنفس الوقت</td><td data-bbox="519 1774 917 1837">يضمن إنك شغال على آخر نسخة</td></tr> <tr> <td data-bbox="1104 1858 1624 1921"><code>git pull</code> قبل ما تبلىش شغل جديد</td><td data-bbox="406 1858 917 1921">أسهل بتتبع الأخطاء والرجوع لنقطة معينة</td></tr> <tr> <td data-bbox="1096 1942 1624 2005">commits صغيرة ومتكررة برسائل واضحة</td><td data-bbox="292 1942 917 2005">يسمح بمراجعة الكود قبل ما يدخل المشروع الرئيسي</td></tr> <tr> <td data-bbox="1128 2026 1624 2089">Pull Request بدل الدفع المباشر لـ <code>main</code></td><td data-bbox="503 2026 917 2089">يمنع كسر الكود على باقي الفريق</td></tr> <tr> <td data-bbox="1128 2026 1624 2089">حل التعارضات محلياً وتجربتها قبل الدفع</td><td></td></tr> </table>		الممارسة	ليش مهمة	كل واحد يشتغل على فرع خاص (<code>feature/اسمك-المهمة</code>) يمنع تعارض شغل الكل مع بعض بنفس الوقت	يضمن إنك شغال على آخر نسخة	<code>git pull</code> قبل ما تبلىش شغل جديد	أسهل بتتبع الأخطاء والرجوع لنقطة معينة	commits صغيرة ومتكررة برسائل واضحة	يسمح بمراجعة الكود قبل ما يدخل المشروع الرئيسي	Pull Request بدل الدفع المباشر لـ <code>main</code>	يمنع كسر الكود على باقي الفريق	حل التعارضات محلياً وتجربتها قبل الدفع	
الممارسة	ليش مهمة												
كل واحد يشتغل على فرع خاص (<code>feature/اسمك-المهمة</code>) يمنع تعارض شغل الكل مع بعض بنفس الوقت	يضمن إنك شغال على آخر نسخة												
<code>git pull</code> قبل ما تبلىش شغل جديد	أسهل بتتبع الأخطاء والرجوع لنقطة معينة												
commits صغيرة ومتكررة برسائل واضحة	يسمح بمراجعة الكود قبل ما يدخل المشروع الرئيسي												
Pull Request بدل الدفع المباشر لـ <code>main</code>	يمنع كسر الكود على باقي الفريق												
حل التعارضات محلياً وتجربتها قبل الدفع													

(10) ملخص سريع — تسلسل يوم عمل عادي



```
bash
git checkout main
git pull                # جيب آخر تحديثات
git checkout -b feature/ # افتح فرع جديد لشغلتك/مهمتي
# ... اشتغل وعدل الملفات ...
git add .
git commit -m "وصف واضح للتعديل"
git push -u origin feature/ # ادفع الفرع لأول مرة/مهمتي
# افتح Pull Request على GitHub
# بعد الموافقة والدمج:
git checkout main
git pull                # جيب النسخة المدموجة الجديدة
git branch -d feature/ # احذف الفرع المحلي (خلص شغلو)/مهمتي
```